Plan de implementación con Claude Code — Recomendador de proveedores

Oct 23, 2026 · @Juanis

Cómo usar este plan

El plan construye el MVP en siete pasos pequeños, cada uno con un prompt listo para pegar en Claude Code y un criterio de aceptación para saber si quedó bien. Sigue el informe por fases: input, transformación y output, sin ajuste por cuotas ni machine learning.

- **Un paso por vez.** Pega el prompt, deja que Claude Code proponga su plan, apruébalo y revisa el resultado antes de pasar al siguiente.
- **Pide pruebas siempre.** Cada prompt exige pruebas automáticas; no avances si fallan.
- **Todo con datos ficticios.** Los archivos de ejemplo se pueden reemplazar después por datos reales sin cambiar el código, si se respetan las mismas columnas.
- **Haz un commit al cerrar cada paso**, para poder volver atrás si algo se rompe.

Paso 0 · Preparar el proyecto

Stack propuesto: Python 3.11 o superior, pandas para los datos, pydantic para validar solicitudes, PyYAML para la configuración, pytest para las pruebas y Streamlit para el dashboard. Los datos viven en archivos CSV; no hace falta una base de datos todavía.

```
recomendador-proveedores/
├── CLAUDE.md                # contexto del proyecto para Claude Code
├── README.md
├── requirements.txt
├── config/
│   └── perfiles.yaml        # pesos por perfil y parámetros
├── data/
│   ├── solicitudes_ejemplo.csv
│   ├── proveedores.csv
│   ├── rutas_proveedor.csv
│   ├── tarifas.csv
│   ├── capacidad.csv
│   ├── historial_viajes.csv
```

```
|   └── etiquetas.csv
├── src/
|   ├── solicitud.py         # paso 3
|   ├── restricciones.py     # paso 4
|   ├── scoring.py           # paso 5
|   └── ranking.py           # paso 6
├── app.py                   # dashboard (paso 6)
└── tests/
```

Prompt para Claude Code:

```
Vamos a construir un MVP en Python que recomienda qué proveedor de transporte
terrestre
de contenedores debe atender cada solicitud, para el equipo de operaciones de
una naviera.
El flujo es: cargar solicitud -> descartar proveedores que no cumplen
restricciones ->
puntuar a los elegibles con pesos configurables por perfil -> ranking top 3 y
un dashboard simple.

En este paso solo prepara el proyecto:
1. Crea la estructura de carpetas: config/, data/, src/, tests/ y archivos
app.py, README.md.
2. Crea requirements.txt con pandas, pydantic, pyyaml, pytest y streamlit.
3. Crea CLAUDE.md con: objetivo del proyecto, el flujo anterior, convenciones
   (código y comentarios en español, funciones pequeñas y puras, toda regla
de negocio con prueba,
   nada de cuotas contractuales ni machine learning en el MVP) y cómo correr
pruebas y la app.
4. No escribas lógica de negocio todavía.
Primero muéstrame tu plan y espera mi aprobación.
```

Criterio de aceptación: la estructura existe, `pip install -r requirements.txt` funciona y `pytest` corre sin errores (aunque aún no haya pruebas).

Paso 1 · Input: documento de ejemplo de requisitos del envío

El primer entregable es un CSV con solicitudes ficticias que cubren los casos normales y los difíciles. Cada fila es una solicitud; las columnas son los campos del input definidos en el informe.

Columna	Tipo	Valores válidos
id_solicitud	texto	SOL-0001, SOL-0002…
origen	código	BUN-SPRBUN, BUN-TCBUEN, CTG-CONTECAR, CTG-SPRC, BAQ-SPRB
destino	código	BOG, MDE, CLO, BGA, PEI, BAQ
fecha_cita	fecha y hora	ISO, p. ej. 2026-11-04T06:00
tipo_contenedor	código	20DV, 40DV, 40HC, 20RF, 40RF, 40FR, 40OT
peso_bruto_t	número	toneladas, incluye el contenedor
imo	sí/no	true / false
clase_imo	número o vacío	1 a 9; obligatoria si imo = true
refrigerada	sí/no	true / false
temperatura_c	número o vacío	obligatoria si refrigerada = true
extradimensionada	sí/no	true / false
valor_declarado_usd	número	mayor que 0
modalidad_aduanera	código o vacío	NACIONALIZADA, DTA, ZONA_FRANCA
prioridad	código	normal, urgente, economica, alto_valor
observaciones	texto o vacío	libre

```
id_solicitud,origen,destino,fecha_cita,tipo_contenedor,peso_bruto_t,imo,clase
_imo,refrigerada,temperatura_c,extradimensionada,valor_declarado_usd,modalida
d_aduanera,prioridad,observaciones
SOL-0001,BUN-SPRBUN,BOG,2026-11-
04T06:00,40HC,26,false,,false,,false,85000,DTA,normal,
SOL-0002,BUN-SPRBUN,BOG,2026-11-
04T08:00,40HC,24,false,,false,,false,60000,NACIONALIZADA,urgente,Cliente
espera en planta
SOL-0003,CTG-CONTECAR,MDE,2026-11-
05T05:00,40RF,28,false,,true,-18,false,120000,NACIONALIZADA,normal,Cadena de
frio
SOL-0004,BUN-TCBUEN,CLO,2026-11-
05T10:00,20DV,18,true,3,false,,false,40000,NACIONALIZADA,normal,
SOL-0005,BAQ-SPRB,BGA,2026-11-
06T07:00,40DV,22,false,,false,,false,350000,ZONA_FRANCA,alto_valor,Requiere
GPS en linea
```

Prompt para Claude Code:

```
Paso 1 del plan (ver CLAUDE.md). Crea data/solicitudes_ejemplo.csv con 12
solicitudes ficticias
y estas columnas exactas, en este orden:
id_solicitud, origen, destino, fecha_cita, tipo_contenedor, peso_bruto_t,
imo, clase_imo,
refrigerada, temperatura_c, extradimensionada, valor_declarado_usd,
modalidad_aduanera,
prioridad, observaciones.

Valores válidos:
- origen: BUN-SPRBUN, BUN-TCBUEN, CTG-CONTECAR, CTG-SPRC, BAQ-SPRB
- destino: BOG, MDE, CLO, BGA, PEI, BAQ
- tipo_contenedor: 20DV, 40DV, 40HC, 20RF, 40RF, 40FR, 40OT
- modalidad_aduanera: NACIONALIZADA, DTA, ZONA_FRANCA o vacío
- prioridad: normal, urgente, economica, alto_valor
- fechas en ISO (2026-11-04T06:00), todas en noviembre de 2026

Incluye al menos: 4 del corredor BUN-SPRBUN -> BOG (para comparar
proveedores), 2 refrigeradas,
1 IMO, 1 extradimensionada en 40FR, 1 de alto valor (> 300000 USD), 2
urgentes y 1 económica.
Crea también data/solicitudes_invalidas.csv con 5 filas que violen una regla
cada una
(peso excesivo, IMO sin clase, refrigerada sin temperatura, origen
```

```
desconocido, fecha mal escrita),
para probar validaciones en el paso 3.
Guarda las listas de valores válidos en config/catalogos.yaml para
reutilizarlas.
```

Criterio de aceptación: los dos CSV abren en Excel sin columnas corridas, y cada caso difícil pedido aparece al menos una vez.

Paso 2 · Input: base de datos de ejemplo con las variables

La "base de datos" del MVP son siete archivos CSV y un YAML, uno por cada tipo de dato. Separarlos así permite actualizar, por ejemplo, las tarifas sin tocar la ficha del proveedor.

Archivo	Una fila por	Columnas	Variable que alimenta
proveedores.csv	proveedor	id_proveedor, nombre, equipos (lista separada por |), peso_max_t, permite_imo, permite_refrigerada, permite_extradimensionada, poliza_monto_usd, poliza_vence, habilitacion_vence	Restricciones
rutas_proveedor.csv	proveedor y ruta	id_proveedor, origen, destino	Restricción de cobertura
tarifas.csv	proveedor, ruta y equipo	id_proveedor, origen, destino, tipo_contenedor, precio_cop	Precio
capacidad.csv	proveedor, fecha y equipo	id_proveedor, fecha, tipo_contenedor, estado (confirmada / sin_capacidad), camiones_libres	Restricción de capacidad; si no hay fila, es desconocida
historial_viajes.csv	viaje realizado	id_viaje, id_proveedor, origen, destino, tipo_contenedor, cargue, entrega, entrega_comprometida, novedad	Confiabilidad y tiempo de tránsito

Archivo	Una fila por	Columnas	Variable que alimenta
etiquetas.csv	etiqueta	id_proveedor, tipo (fortaleza / debilidad), etiqueta, condicion	Fortalezas y debilidades
tiempos_base.csv	ruta	origen, destino, horas_base	Tiempo de tránsito sin historial (reemplaza a la API de mapas en el MVP)
config/perfiles.yaml	—	pesos por perfil, piso de confiabilidad, mínimo de viajes, ajuste por etiqueta	Scoring

La columna `condicion` de etiquetas dice cuándo aplica la etiqueta, con la forma `campo=valor`: por ejemplo `refrigerada=true`, `origen=BUN-*` o `prioridad=alto_valor`.

```
# config/perfiles.yaml
perfiles:
  normal:     {precio: 0.35, tiempo: 0.15, confiabilidad: 0.35, etiquetas:
0.15}
  urgente:    {precio: 0.15, tiempo: 0.35, confiabilidad: 0.35, etiquetas:
0.15}
  economica:  {precio: 0.55, tiempo: 0.10, confiabilidad: 0.25, etiquetas:
0.10}
  alto_valor: {precio: 0.20, tiempo: 0.15, confiabilidad: 0.35, etiquetas:
0.30}
parametros:
  piso_confiabilidad: 0.80     # bajo este % a tiempo, el puntaje es 0
  min_viajes: 20               # con menos viajes se mezcla con el promedio
general
  etiqueta_base: 0.5
  ajuste_por_etiqueta: 0.25
```

Prompt para Claude Code:

```
Paso 2 del plan. Crea los datos ficticios de proveedores en data/ y la
configuración en config/perfiles.yaml,
con exactamente estos archivos y columnas:
- proveedores.csv: id_proveedor, nombre, equipos (lista separada por |),
```

```
peso_max_t, permite_imo,
  permite_refrigerada, permite_extradimensionada, poliza_monto_usd,
poliza_vence, habilitacion_vence
- rutas_proveedor.csv: id_proveedor, origen, destino
- tarifas.csv: id_proveedor, origen, destino, tipo_contenedor, precio_cop
- capacidad.csv: id_proveedor, fecha, tipo_contenedor, estado (confirmada |
sin_capacidad), camiones_libres
  (si no hay fila para un proveedor y fecha, la capacidad es desconocida)
- historial_viajes.csv: id_viaje, id_proveedor, origen, destino,
tipo_contenedor, cargue, entrega,
  entrega_comprometida, novedad
- etiquetas.csv: id_proveedor, tipo (fortaleza | debilidad), etiqueta,
condicion (campo=valor, admite * al final)
- tiempos_base.csv: origen, destino, horas_base
- config/perfiles.yaml: perfiles normal, urgente, economica y alto_valor con
pesos de precio, tiempo,
  confiabilidad y etiquetas que sumen 1; parametros piso_confiabilidad 0.80,
min_viajes 20,
  etiqueta_base 0.5, ajuste_por_etiqueta 0.25.
Pesos por perfil (precio/tiempo/confiabilidad/etiquetas): normal
0.35/0.15/0.35/0.15,
urgente 0.15/0.35/0.35/0.15, economica 0.55/0.10/0.25/0.10, alto_valor
0.20/0.15/0.35/0.30.
Usa los códigos de config/catalogos.yaml. Proveedores ficticios P01 a P06,
con nombres inventados
(no uses nombres de empresas reales).

Diseña los datos para que el corredor BUN-SPRBUN -> BOG con 40HC el 2026-11-
04 produzca este caso de prueba:
- P01: precio 4.600.000 COP, mediana de tránsito 24 h, 92% a tiempo, sin
etiquetas que coincidan.
- P02: precio 5.000.000, mediana 20 h, 97% a tiempo, una fortaleza que
coincide con esa solicitud.
- P03: precio 4.400.000, mediana 28 h, 85% a tiempo, una debilidad con
condicion origen=BUN-*.
- P04: cubre la ruta pero tiene estado sin_capacidad ese día.
- P01, P02 y P03 con 100 viajes cada uno en ese corredor en
historial_viajes.csv.
Además: P05 especializado en refrigerada, P06 con permiso IMO pero póliza
baja,
```

```
un proveedor con habilitación vencida y uno nuevo con solo 3 viajes.
Agrega un script src/verificar_datos.py que revise que todos los CSV tienen
las columnas esperadas
y que los códigos existen en el catálogo.
```

Criterio de aceptación: `python src/verificar_datos.py` termina sin errores, y en el historial los porcentajes a tiempo y las medianas de P01, P02 y P03 dan exactamente los valores pedidos.

Paso 3 · Transformación: carga de requisitos como solicitud

Este paso convierte cada fila del CSV en un objeto `Solicitud` validado. Una fila con errores no detiene la carga: se aparta con el motivo, para que el operario la corrija.

Prompt para Claude Code:

```
Paso 3 del plan. Crea src/solicitud.py con:
1. Un modelo pydantic Solicitud con todos los campos de
data/solicitudes_ejemplo.csv y tipos correctos
   (fecha_cita como datetime, booleanos reales, números como float).
2. Validaciones:
   - origen, destino, tipo_contenedor, modalidad_aduanera y prioridad deben
existir en config/catalogos.yaml
   - peso_bruto_t no puede superar el máximo del tipo de contenedor (agrega
peso_max_t por tipo en catalogos.yaml)
   - si imo es true, clase_imo es obligatoria (1 a 9)
   - si refrigerada es true, temperatura_c es obligatoria y el contenedor
debe ser RF
   - si extradimensionada es true, el contenedor debe ser FR u OT
   - valor_declarado_usd mayor que 0; prioridad vacía equivale a normal
3. Una función cargar_solicitudes(ruta_csv) que devuelva
(solicitudes_validas, errores), donde cada error
   tiene: fila, id_solicitud, campo y mensaje en español entendible para un
operario.
4. Pruebas en tests/test_solicitud.py:
   - solicitudes_ejemplo.csv carga 12 válidas y 0 errores
   - solicitudes_invalidas.csv da 0 válidas y exactamente un error por fila,
con el campo correcto
No toques todavía restricciones ni scoring.
```

Criterio de aceptación: `pytest tests/test_solicitud.py` pasa, y los mensajes de error se entienden sin leer el código (por ejemplo: "Fila 3: la carga es refrigerada pero no tiene temperatura").

Paso 4 · Transformación: comparación con las restricciones de cada proveedor

Cada restricción es una función pequeña que recibe una solicitud y un proveedor y responde si pasa y por qué no. El resultado separa a los proveedores en elegibles, elegibles con alerta y descartados.

Restricción	Pasa si…	Motivo si no pasa
Cobertura	La ruta origen–destino está en rutas_proveedor.csv	"No cubre BUN-SPRBUN → BOG"
Equipo	El tipo de contenedor está en su lista de equipos	"No tiene equipo 40RF"
Peso	peso_bruto_t ⩽ peso_max_t del proveedor	"Peso 31 t supera su máximo de 30 t"
IMO	Si la carga es IMO, permite_imo = true	"No está habilitado para carga IMO"
Refrigerada	Si es refrigerada, permite_refrigerada = true	"No maneja carga refrigerada"
Extradimensionada	Si lo es, permite_extradimensionada = true	"No maneja carga extradimensionada"
Póliza	poliza_monto_usd ⩾ valor declarado	"Póliza de USD 200.000 no cubre USD 350.000"
Documentos	Póliza y habilitación vigentes en la fecha de la cita	"Habilitación vencida el 2026-10-01"
Tarifa	Existe precio para esa ruta y equipo	"Sin tarifa para esta ruta y equipo"
Capacidad	Estado confirmada (pasa) o sin fila (pasa con alerta)	"Sin capacidad el 2026-11-04"

La tarifa se trata como restricción porque sin precio no se puede puntuar. Los descartados por una sola regla se marcan como "casi elegibles" para mostrarlos si nadie pasa.

Prompt para Claude Code:

```
Paso 4 del plan. Crea src/restricciones.py.
1. Una función cargar_datos() que lea los CSV de proveedores, rutas, tarifas
y capacidad de data/.
2. Una función por restricción, cada una con firma (solicitud, proveedor,
datos) -> ResultadoRegla(pasa: bool,
   alerta: str | None, motivo: str | None). Restricciones: cobertura, equipo,
peso, imo, refrigerada,
   extradimensionada, poliza, documentos vigentes a la fecha de la cita,
tarifa existente y capacidad.
   Capacidad tiene tres estados: confirmada pasa; sin_capacidad no pasa; sin
fila pasa con alerta
   "Capacidad por confirmar".
3. Una función evaluar_proveedores(solicitud, datos) que aplique todas las
reglas a todos los proveedores
   y devuelva: elegibles (con sus alertas), descartados (con todos sus
motivos) y casi_elegibles
   (descartados que fallaron una sola regla).
4. Mensajes en español con datos concretos, por ejemplo "Póliza de USD
200.000 no cubre USD 350.000".
5. Pruebas en tests/test_restricciones.py con una prueba por regla (un caso
que pasa y uno que no) y:
   - SOL del corredor BUN-SPRBUN -> BOG 40HC del 2026-11-04: elegibles P01,
P02 y P03; P04 descartado por capacidad
   - el proveedor con habilitación vencida nunca es elegible
   - una solicitud refrigerada deja elegible a P05
No hagas scoring todavía.
```

Criterio de aceptación: pasan las pruebas, y para cualquier solicitud se puede imprimir una tabla proveedor por proveedor con el resultado de cada regla.

Paso 5 · Transformación: perfiles de pesos y generación del score

El scoring solo recibe a los elegibles del paso 4. Convierte cada variable a una escala de 0 a 1, aplica los pesos del perfil que indica la prioridad de la solicitud y devuelve un puntaje de 0 a 100 con su desglose.

Variable	De dónde sale el valor	Cómo se lleva a 0–1
Precio	tarifas.csv para esa ruta y equipo	(máx − valor) / (máx − mín) entre elegibles; si todos son iguales o hay uno solo, 1
Tiempo	Mediana de horas (entrega − cargue) del proveedor en ese corredor si tiene al menos min_viajes; si no, horas_base	Igual que el precio
Confiabilidad	% de viajes con entrega ⩽ entrega_comprometida en ese corredor (o en todo su historial si ahí tiene pocos); con menos de min_viajes en total, se mezcla con el promedio de todos	(% − piso) / (1 − piso), limitado entre 0 y 1
Etiquetas	etiquetas.csv cuya condición coincide con la solicitud	0,5 + 0,25 por fortaleza − 0,25 por debilidad, limitado entre 0 y 1

La prueba central de este paso reproduce el ejemplo del informe. Si estos números salen, el scoring está bien implementado.

Proveedor	Perfil normal	Perfil urgente
P01	59,33	56,00
P02	56,00	76,00
P03	47,50	27,50

Prompt para Claude Code:

```
Paso 5 del plan. Crea src/scoring.py.
1. Lee pesos y parámetros de config/perfiles.yaml; valida que los pesos de
cada perfil sumen 1.
2. Para cada proveedor elegible calcula los valores crudos:
   - precio: de tarifas.csv para la ruta y el tipo de contenedor
   - tiempo: mediana de horas (entrega - cargue) del proveedor en ese origen-
destino si tiene al menos
     min_viajes viajes; si no, horas_base de tiempos_base.csv
   - confiabilidad: fracción de viajes con entrega <= entrega_comprometida en
```

```
ese origen-destino
     (si ahí tiene menos de min_viajes, usar todo su historial);
     solo si tiene menos de min_viajes, mezclar: (aciertos + min_viajes *
promedio_general) / (viajes + min_viajes)
   - etiquetas: lista de etiquetas cuya condicion (campo=valor, con * como
comodín al final) coincide con la solicitud
3. Normaliza a 0-1:
   - precio y tiempo: (max - valor) / (max - min) entre los elegibles; 1 si
todos iguales o hay uno solo
   - confiabilidad: (valor - piso) / (1 - piso), limitado a [0, 1]
   - etiquetas: etiqueta_base + ajuste * fortalezas - ajuste * debilidades,
limitado a [0, 1]
4. Puntaje = 100 * suma(peso * valor_normalizado), con el perfil = prioridad
de la solicitud.
5. Devuelve por proveedor: puntaje, valores crudos, valores normalizados,
aporte de cada variable
   (peso * normalizado * 100) y las etiquetas que coincidieron.
6. Pruebas en tests/test_scoring.py:
   - con SOL-0001 (normal): P01 59.33, P02 56.00, P03 47.50 (tolerancia 0.01)
   - la misma solicitud con prioridad urgente: P01 56.00, P02 76.00, P03
27.50
   - un solo elegible obtiene 1 en precio y tiempo
   - el proveedor con 3 viajes usa confiabilidad mezclada y horas_base
   - cambiar un peso en el YAML cambia el puntaje sin tocar código
```

Criterio de aceptación: pasan las pruebas, en especial las dos del ejemplo del informe, y el desglose suma exactamente el puntaje total.

Paso 6 · Output: ranking y dashboard sencillo

El ranking ordena a los elegibles por puntaje y arma una explicación corta para cada uno. El dashboard solo muestra eso: no asigna, no guarda decisiones y no tiene indicadores de seguimiento todavía.

Contenido del dashboard, nada más:

1. **Selector de solicitud** con las solicitudes válidas del CSV, y un aviso con las filas que no cargaron y su motivo.
2. **Resumen de la solicitud:** ruta, contenedor, fecha de cita, prioridad y perfil de pesos aplicado.

3. **Top 3:** tabla con posición, proveedor, puntaje, precio, tiempo estimado, % a tiempo, alertas y una frase de explicación.
4. **Desglose:** barras apiladas con el aporte de precio, tiempo, confiabilidad y etiquetas para cada proveedor del top 3.
5. **Descartados:** sección plegable con cada proveedor descartado y sus motivos; si no hay elegibles, se muestran primero los casi elegibles.
6. **Vista general:** una tabla con todas las solicitudes y el proveedor recomendado para cada una.

Prompt para Claude Code:

```
Paso 6 del plan. Crea src/ranking.py y app.py.

src/ranking.py:
1. recomendar(solicitud, datos) que encadene evaluar_proveedores (paso 4) y
el scoring (paso 5) y devuelva:
   top 3 ordenado por puntaje (desempate: mayor confiabilidad, luego menor
precio), todos los elegibles,
   descartados, casi_elegibles y alertas.
2. Una explicación en español de máximo 20 palabras por proveedor, generada
con reglas a partir de su
   variable de mayor aporte y su alerta, por ejemplo "El más rápido y
confiable del corredor; capacidad por confirmar".
3. Pruebas en tests/test_ranking.py: SOL-0001 da el orden P01, P02, P03; con
prioridad urgente da P02, P01, P03;
   una solicitud sin elegibles devuelve top vacío y casi_elegibles con
motivo.

app.py (Streamlit, una sola página, sin login ni base de datos):
- selector de solicitud y aviso con los errores de carga
- resumen de la solicitud y perfil aplicado
- tabla top 3: posición, proveedor, puntaje, precio, tiempo estimado, % a
tiempo, alertas, explicación
- barras apiladas con el aporte de cada variable por proveedor del top 3
- expander con descartados y motivos (si no hay elegibles, mostrar
casi_elegibles arriba)
- una tabla general con todas las solicitudes y su proveedor recomendado
No agregues asignación, registro de decisiones ni otros indicadores.
Actualiza README.md con cómo instalar, correr pruebas (pytest) y abrir la app
(streamlit run app.py).
```

Criterio de aceptación: `streamlit run app.py` abre el dashboard; al elegir SOL-0001 aparece P01 primero con 59 puntos, y al cambiar a una solicitud urgente del mismo corredor gana P02.

Verificación final

Al terminar el paso 6, una última sesión revisa el MVP completo antes de mostrarlo al equipo.

Prompt para Claude Code:

```
Revisión final del MVP. Sin agregar funcionalidades:
1. Corre todas las pruebas y corrige lo que falle.
2. Recorre el flujo completo con las 12 solicitudes de ejemplo y muéstrame
una tabla con:
   solicitud, prioridad, número de elegibles, recomendado, puntaje y alertas.
3. Revisa que ninguna regla de negocio esté escrita a mano fuera de config/
(pesos, pisos, catálogos).
4. Lista los supuestos que tomaste y cualquier caso que el código no maneje
bien.
```

Lista de chequeo antes de mostrarlo:

- [ ] `pytest` pasa completo.
- [ ] El ejemplo del informe da 59 / 56 / 48 con perfil normal y 56 / 76 / 28 con perfil urgente.
- [ ] Cambiar un peso en `config/perfiles.yaml` cambia el ranking sin tocar código.
- [ ] Las solicitudes inválidas muestran mensajes entendibles.
- [ ] Una solicitud sin elegibles muestra los casi elegibles con su motivo.
- [ ] Reemplazar un CSV de ejemplo por otro con las mismas columnas no rompe nada.

Siguiente paso natural después del MVP: cargar datos reales de un mes y comparar las recomendaciones con lo que los operarios decidieron en realidad.